

FP_DA2_0A01 – Programació Avançada (Angular)

Unitat 3 – Serveis i gestió de dades (RA3)

EAC3

(Curs 2025–26 / 2n semestre)

Espai de Resposta de l'alumne

Instruccions: Abans de lliurar aquest document a la tasca, omple obligatòriament els següents camps amb la informació del teu projecte.

Nom complet de l'alumne:

ACHRAF BEN TAMOU

Nom de l'aplicació (carpeta del projecte):

PCTrade

Breu descripció funcional (Què farà la teva app?):

Plataforma orientada a la economía circular y la sostenibilidad que permite a los usuarios intercambiar, comprar o vender componentes de hardware de PC de segunda mano para reducir la basura electrónica.

URL del repositori GitHub (pot ser públic):

<https://github.com/AshNinjaYT/PCTrade>

URL de la branca de treball corresponent de GitHub:

<https://github.com/AshNinjaYT/PCTrade/tree/ra3-serveis>

Exercici 1: Serveis: HTTP + estats de càrrega / èxit / error (2,5 punts)

Enunciat: Implementeu el servei `ElementService` al fitxer `src/app/serveis/element.service.ts`. Aquest servei és el nucli de comunicació entre l'aplicació i l'API mock, i ha de gestionar els tres estats de manera reactiva amb signals.

El servei ha d'exposar les signals de lectura següents:

- `elements`: `Signal<ElementCatalog[]>` — llista d'elements carregats des de l'API.
- `carregant`: `Signal<boolean>` — `true` mentre hi ha una petició HTTP en curs.
- `error`: `Signal<string | null>` — missatge d'error descriptiu, o `null` si tot va bé.

El servei ha d'implementar els mètodes:

- `obtenirPopulars()`: `void` — carrega els elements amb `popular=true` des de l'API.
- `cercar(terme: string)`: `void` — filtra elements per nom consultant l'API.

Requisit important: cada mètode ha de gestionar els tres estats: activar `carregant=true` en iniciar la petició, actualitzar elements en cas d'èxit, i establir un missatge d'error comprensible en cas de fallada.

Heu de configurar `provideHttpClient()` a `app.config.ts` i definir la URL base de l'API en un fitxer d'entorn (`src/environments/environment.ts`).

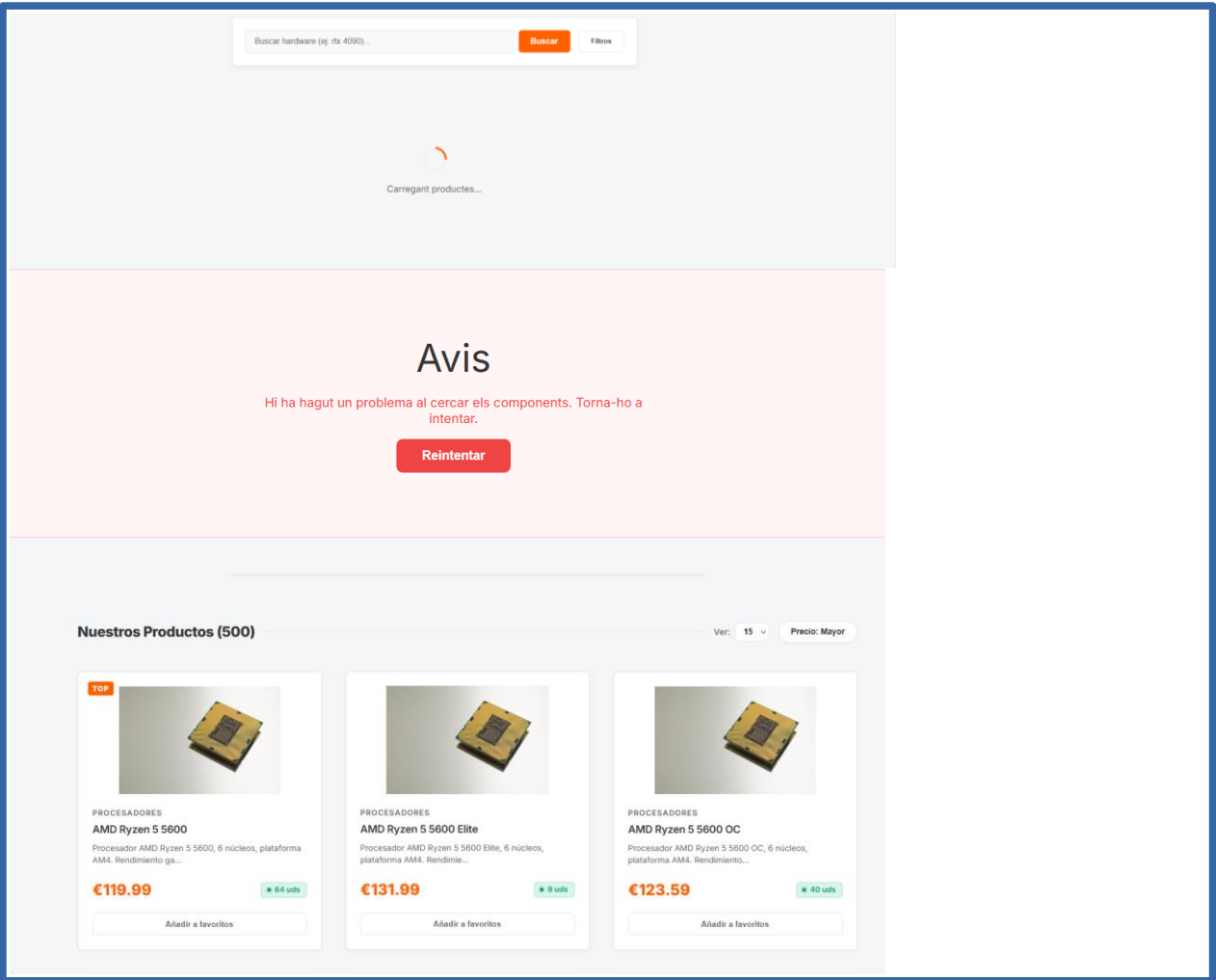
El component `CatalogPage` ha de mostrar els tres estats diferenciats visualment:

- **Carregant:** spinner animat mentre l'API respon.
- **Error:** missatge clar i botó «Reintentar».
- **Èxit:** graella responsiva amb les targetes dels elements.

Captura requerida 1: *Estat carregant — spinner visible mentre l'API respon (retard de 600 ms del json-server).*

Captura requerida 2: *Estat error — missatge d'error i botó «Reintentar» visibles (atureu el json-server per simular-ho).*

Captura requerida 3: *Estat èxit — graella d'elements carregats correctament des de l'API.*



Rúbrica	A (2.50p)	B (2.00p)	C (1.25p)	D (0.5p)	E (0.00p)
Serveis: HTTP + estats de càrrega / èxit / error	ElementService complet amb HttpClient, signals asReadOnly() i els tres estats gestionats i visualitzats. Missatges d'error comprensibles. Captures adjuntades.	Servei funcional però falta gestionar algun estat (p.ex. carregant no s'activa o error no mostra missatge). Captures adjuntades.	El servei carrega dades però no gestiona errors o els estats no s'actualitzen correctament. Manca alguna captura.	Servei parcialment implementat: errors de tipus o peticions HTTP incorrectes. Signals no configurades.	No realitzat.

Exercici 2: Models/Adaptadors: Estructures i transformació de dades (2 punts)

Enunciat: Definiu les estructures de dades que l'aplicació necessita per treballar amb l'API de forma tipada i segura.

Heu de crear al fitxer `src/app/models/element.model.ts` dues interfícies TypeScript separades:

- `ElementApiResponse` — representa exactament l'estructura JSON que retorna l'API (camps originals).
- `ElementCataleg` — representa el model intern de l'aplicació (noms de camps en català, tipus adaptats).

Heu de crear al fitxer `src/app/adaptadors/element.adaptador.ts` dues funcions exportades:

- `adaptarElementApi(apiResponse: ElementApiResponse): ElementCataleg` — transforma un sol element.
- `adaptarElementsApi(apiResponses: ElementApiResponse[]): ElementCataleg[]` — transforma un array.

Camps mínims requerits (mapeig API → model intern):

ElementApiResponse	ElementCataleg	Tipus
id	id	string
nom	titol	string
descripcio	descripcio	string
categoria	categoria	string
preu	preu	number
imatge	imatgeUrl	string
popular	esPopular	boolean
stock	unitats	number

Documentació obligatòria a `docs/models.md`: incloeu la taula de mapeig de camps anterior amb el nom original de l'API, el nom intern i el tipus TypeScript.



Rúbrica	A (2.00p)	B (1.50p)	C (1.00p)	D (0.50p)	E (0.00p)
Models/Adaptadors	Dues interfícies correctament definides amb tots els camps i tipus coherents. Adaptadors implementats i funcionals. Mapeig documentat a docs/models.md..	Interfícies correctes però falta algun camp o tipatge imprecís. Adaptadors funcionals. Documentació incompleta.	Interfícies definides però falten camps o no s'usa interface correctament. Adaptadors parcials o amb errors de tipus.	Errors de sintaxi o estructura incompleta. Adaptadors no funcionals o absents.	No realitzat.

Exercici 3: Formularis: Reactius + validacions sync/async (2 punts)

Enunciat: Implementeu el component `FormulariCerca` al fitxer `src/app/components/formulari-cerca/`. Aquest component ha de gestionar la cerca d'elements amb un formulari reactiu i validacions avançades.

El formulari ha d'incloure el camp `termCerca` amb:

- Validador síncron: longitud mínima de 2 caràcters i màxima de 50.
- Validador asíncron `codiDisponibleValidator`: simula una consulta a l'API per comprovar si hi ha resultats (retard de 500 ms). Ha de retornar `{ sensResultats: true }` si no hi ha elements.

Comportament del component:

- Cerca automàtica amb `debounce` de 400 ms mentre l'usuari escriu.
- Botó «Netejar» que apareix únicament quan el camp conté text.
- Indicador visual («Validant...» o spinner) durant la validació asíncrona.
- Missatges d'error visibles per a cadascuna de les validacions.
- Els errors no s'han de mostrar fins que l'usuari hagi tocat el camp (`ng-touched`).

Documentació obligatòria a `docs/formularis.md`: descriuiu els validadors implementats, síncrons i asíncrons, i el comportament del `debounce`.

Captura requerida 4: *Formulari amb missatge d'error de validació síncron visible (menys de 2 caràcters).*

Captura requerida 5: *Formulari amb indicador de validació asíncrona en curs*

The image shows two screenshots of the `FormulariCerca` component. The top screenshot shows a search input field with the text 'a'. Below the field, a red error message is displayed: 'El terme de cerca ha de tenir com a mínim 2 caràcters.' The bottom screenshot shows the same search input field with the text 'amd'. Below the field, an orange indicator 'Validant...' is displayed, indicating that an asynchronous validation is in progress.

Rúbrica	A (2.00p)	B (1.50p)	C (1.00p)	D (0.50p)	E (0.00p)
Formularis : Reactius + validacions sync/async	Formulari complet: validador síncron i asíncron implementats, debounce de 400 ms, botó netejar condicional, errors visuals correctes amb ng-touched. Captures adjuntades.	Formulari funcional però falta el validador asíncron o el debounce no és correcte. Missatges d'error presents. Manca alguna captura.	Validadors síncrons presents però asíncrons absents. Formulari emet la cerca però sense debounce o ng-touched.	Formulari creat però validacions no funcionen, missatges d'error absents o formulari no és reactiu.	No realitzat.

Exercici 4: Preferits: Persistència amb localStorage (1,5 punts)

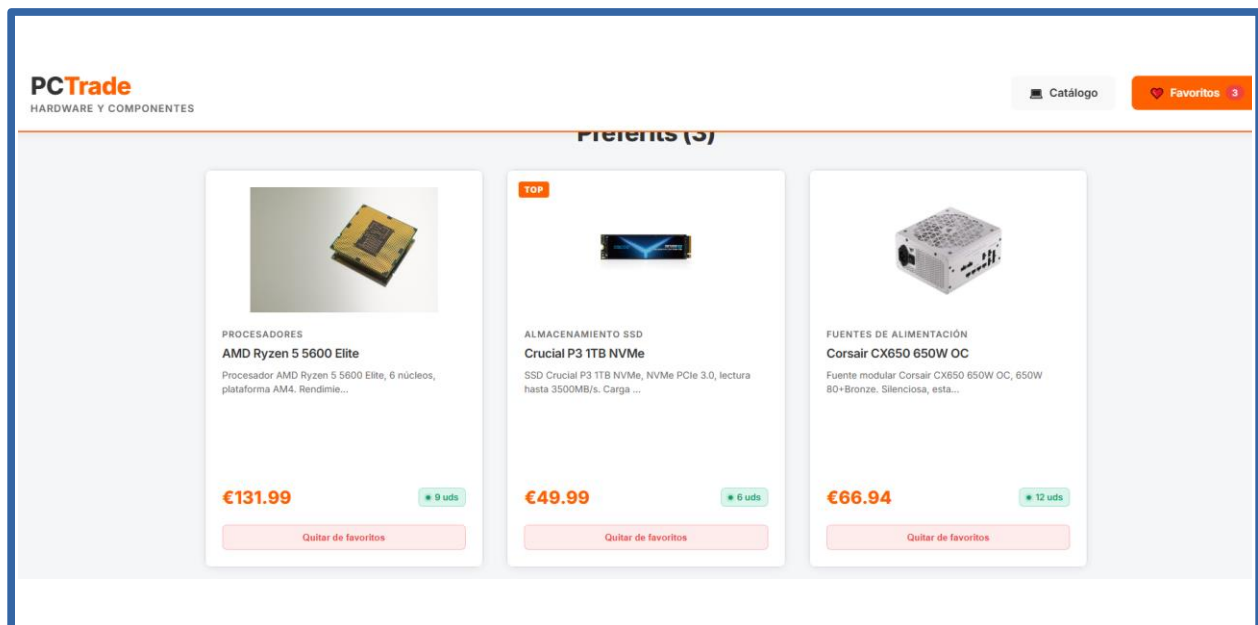
Enunciat: Implementeu el servei PreferitsService al fitxer src/app/serveis/preferits.service.ts per gestionar els elements marcats com a favorits amb persistència entre sessions.

El servei ha d'implementar:

- Clau de localStorage: 'preferits-catalog'.
- Càrrega automàtica dels preferits en inicialitzar el servei.
- Gestió d'errors de localStorage amb try/catch.
- Signals de lectura: preferits (Signal<ElementCatalog[]>) i totalPreferits (computed).
- Mètodes públics:
 - afegirPreferit(element: ElementCatalog): void
 - eliminarPreferit(id: string): void
 - esPreferit(id: string): boolean

Al component element-card, afegiu un Botó (estrella o cor) que cridi afegirPreferit() o eliminarPreferit() segons l'estat actual, i mostri visualment si l'element és preferit.

Captura requerida 6: Panell de preferits amb elements marcats i comptador visible. Els preferits han de persistir en recarregar el navegador (F5).



Rúbrica	A (1.50p)	B (1.00p)	C (0.50p)	D (0.25p)	E (0.00p)
Preferits: Persistència amb localStorage	PreferitsService complet amb signals, computed, mètodes afegir/eliminar/esPreferit. Persistència a localStorage funcional (preferits sobreviuen a F5). Botó a element-card funcional. Captura adjuntada.	Servei funcional però la persistència falla (no carrega en recarregar) o el Botó a la targeta no mostra l'estat visual correctament.	Preferits gestionats en memòria però sense persistència a localStorage, o signals no correctament configurades.	Implementació parcial amb errors significatius. No persisteix o els mètodes no funcionen.	No realitzat.

Exercici 5: FormArray + Docs: Formulari dinàmic i documentació (2 punts)

Enunciat: Implementeu al component preferits-panel un formulari dinàmic amb FormArray per gestionar notes per a cada element preferit, i completeu la documentació tècnica del projecte.

Funcionalitat del panell de preferits:

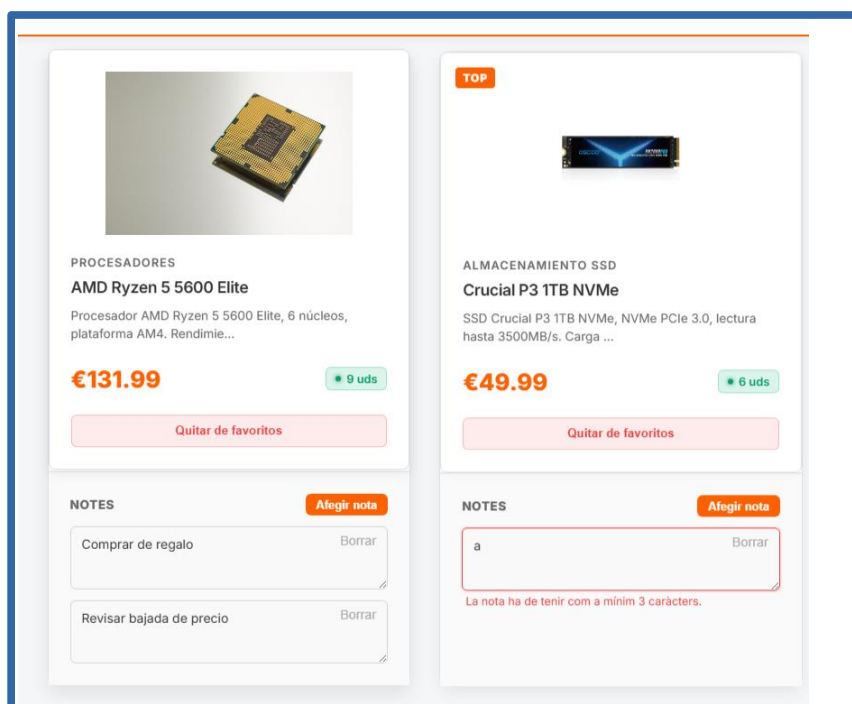
- Llista tots els elements preferits amb les seves notes.
- Per a cada element preferit, un formulari amb FormArray de notes:
 - Botó «+» per afegir un nou camp de text de nota (buit).
 - Botó «x» al costat de cada nota per eliminar-la individualment.
 - Validació de cada nota: mínima 3 caràcters, no pot estar buida.
- Les notes han de persistir conjuntament amb els preferits a localStorage.

Documentació obligatòria a docs/:

- docs/serveis.md — Descripció del servei HTTP (endpoints, mètodes, estats) i del servei de preferits.
- docs/models.md — Interfícies TypeScript i taula de mapeig de camps API → model intern.
- docs/formularis.md — Descripció dels formularis reactius, validadors síncrons i asíncrons, i ús del FormArray.

Captura requerida 7: Panell de preferits amb notes dinàmiques — almenys un element amb 2 notes afegides i els Botons «+» i «x» visibles.

Captura requerida 8: Error de validació de nota visible (nota amb menys de 3 caràcters).



Rúbrica	A (2.00p)	B (1.50p)	C (1.00p)	D (0.50p)	E (0.00p)
FormArray + Docs: Formulari dinàmic i documentació	FormArray implementat: afegir i eliminar notes funcionals, validació de nota (mínim 3 caràcters), notes persisteixen a localStorage. Els tres fitxers de docs/ complets. Captures adjuntades.	FormArray funcional (afegir/eliminar) però les notes no persisteixen correctament, o algun fitxer de docs/ és incomplet.	FormArray creat però sense validació o sense persistència . Documentació parcial (falten fitxers o contingut molt escàs).	Intent de FormArray amb errors significatius . Documentació absent o buida.	No realitzat.